EE-2003
Computer Organization and Assembly Language (COAL)

Week 1

Dr. Shakil Ahmed

Introduction

• Definition 1:

Computer organization is concerned with the way hardware components operate and the way they are connected together to form a computer system.

Definition 2:

• Computer organization is concerned with the structure and behavior of a computer system.

Definition 3:

Computer organization is the study of the physical arrangement, interconnection, and operational behavior of the hardware components of a computer system, which are responsible for executing instructions defined by the computer architecture.

- **Computer Architecture** → *Deals with the functional aspects and logical design of a computer system that are visible to the programmer (e.g., instruction set, addressing modes).*
- **Computer Organization** → *Deals with the physical implementation and operational structure of the computer system (e.g., control signals, data paths, memory technology).*
- ☞ In short:
  Architecture = What a computer does
  Organization = How a computer does it

Definition 4:

• Assembly language consists of statements written with short mnemonics such as ADD, MOV, SUB, and CALL.

Introduction

• The main objective of this course is to: – Understand the basic structure of a computer system – How to write programs by understanding the hardware structure

Assembly Language Programming

• An assembly language is a low-level programming language designed for a specific type of processor

• Assembly language is used to write programs using alphanumeric symbols (or mnemonic), e.g. ADD, MOV, PUSH etc.

• Assembly Language depends on the machine code instructions

– Every assembler has its own assembly language

Assembly Language Programming

• Assembly language has a one-to-one relationship with machine language:

– Each assembly language instruction corresponds to a single machine-language instruction.

– A single machine-language instruction can take up one or more bytes of code.

Assembly Language Programming

- The instruction must specify which operation (opcode) is to be performed and the operands
- E.g. ADD AX, BX
  - ADD is the operation
  - AX is called the destination operand
  - BX is called the source operand
  - The result is AX = AX + BX
- When writing assembly language program, you need to think in the instruction level

Assembly Language Programming Example

- MOV AL, 00H

– The native language of a computer is machine language (using 0,1 to represent the operation)

– The full machine language code (in hexadecimal) for the above instruction is B0 00 (2 bytes)

(B0 is the opcode for MOV AL, imm8)

(00- is the immediate 8 bit value- in this case, 00H)

– After assembled, and linked into an executable program, the value B000 will be stored in the memory

– When the program is executed, then the value B0 00 is read from memory, decoded and carry out the task

- The executable program could be .com, .exe, or .bin files

- Learning assembly language programming will help in understanding the operations of a microprocessor
- To learn:

– Need to know the functions of various registers

– Need to know how external memory is organized and how it is addressed to obtain instructions and data (different addressing modes)

– Need to know what operations (or the instruction set) are supported by the CPU

High Level Languages

- A high-level language (HLL) is a programming language that enables a programmer to write programs that are more or less independent of a particular type of computer
- Such languages are considered high-level because they are closer to human languages
- Examples are C, C++, Java, FORTRAN, or Pascal

High Level Languages

- High-level languages such as C++ and Java have a one-to many relationship with assembly language and machine language.
- A single statement in C++ expands into multiple assembly language or machine instructions.

High Level Languages

- The following C++ code carries out two arithmetic operations and assigns the result to a variable.

Assume X and Y are integers:

Example:

int Y;

int X = (Y + 4) * 3;

- Following is the equivalent translation to assembly language.
- The translation requires multiple statements because assembly language works at a detailed level:

```
mov eax, Y ; move Y to the EAX register
add eax, 4 ; add 4 to the EAX register
mov ebx, 3 ; move 3 to the EBX register
imul ebx ; multiply EAX by EBX
mov X, eax ; move EAX to X
```

- Registers are named storage locations in the CPU that hold intermediate results of operations

What Are Assemblers and Linkers?

- Assembler is a utility program that converts source code programs from assembly language into an object file, a machine language translation of the program. – Optionally a Listing file is also produced.

– We'll use MASM as our assembler.

- The linker reads the object file and checks to see if the program contains any calls to procedures in a link library.

– The linker copies any required procedures from the link library, combines them with the object file, and produces the executable file.

– Microsoft 16-bit linker is LINK.EXE and 32-bit is Linker LINK32.EXE.

- OS Loader: A program that loads executable files into memory, and branches the CPU to the program's starting address, (may initialize some registers (e.g. IP)) and the program begins to execute

- Debugger is a utility program, that lets you step through a program while it's running and examine registers and memory
- MASM provides CodeView,
- TASM provides Turbo Debugger and msdev.exe for 32-bit Window console programs.

Listing File

- A listing file is a text file produced by the assembler that shows the source code, the machine code (object code) generated for each instruction, memory addresses, and sometimes error/warning messages.
- **Contents of a Listing File usually include:**
  - **Line numbers** (from the source code).
  - **Memory addresses** where each instruction is stored.
  - **Object code (machine code)** in hexadecimal.
  - **Original assembly source code** line by line.
  - **Error diagnostics (if any)** during assembly.

- **Purpose of a Listing File:**
  - Helps in **debugging** (checking if the correct machine code was generated).
  - Useful for **understanding how assembly translates to machine instructions**.
  - Acts as a **reference** for locating errors or mismatches.
- Example
- For the assembly code:
  - MOV AX, 5
  - ADD AX, 3
- The listing file may look like:
  - 0000 B80500 MOV AX, 0005
  - 0003 050300 ADD AX, 0003

Here:

- `0000`, `0003` → memory addresses.
- `B80500`, `050300` → machine code in hex.
- `MOV AX, 0005`, `ADD AX, 0003` → original source instructions.

Symbol Table

- A symbol table is a table used by the assembler to keep track of the symbolic names (labels, variables, constants) along with their corresponding memory addresses, types, and other attributes.
- **Contents of a Symbol Table:**
  - **Symbol/Name** → Identifier (like a label or variable).
  - **Address/Location** → Memory location or offset assigned to the symbol.
  - **Type/Attribute** → Data type (byte, word, etc.), scope, or usage.
  - **Value (if constant)** → For EQU or defined constants.

- Example Assembly Code:
  - COUNT DW 10 ; variable COUNT
  - START: MOV AX, COUNT
  - ADD AX, 5

Symbol	Address	Type	Value
COUNT	1000h	WORD	10
START	1002h	LABEL	—

- In most assemblers, the **listing file *does* include the symbol table** (usually printed at the **end of the listing file**).

Detailed Steps

- **Source code** is written by the programmer.
- The **Assembler** processes it in two passes:
  - **Pass 1:** Builds the **symbol table** (assigns addresses to labels/variables).
  - **Pass 2:** Uses the symbol table to generate **machine code**.
- Assembler generates:
  - **Object file (.OBJ / .o):** Binary machine code for linker/loader.
  - **Listing file (.LST):** Human-readable record (source + machine code + symbol table).
  - **Symbol table:** Internal structure, often printed at the end of listing file.

Assembly Language for x86 Processors

- Assembly Language for x86 Processors focuses on programming microprocessors compatible with the Intel IA-32 and AMD x86 processors running under Microsoft Windows.
- Assembly language bears the closest resemblance to native machine language.

Is Assembly Language Portable?

- A language whose source programs can be compiled and run on a wide variety of computer systems is said to be portable.
- A C++ program, for example, should compile and run on just about any computer, unless it makes specific references to library functions that exist under a single operating system.
- A major feature of the Java language is that compiled programs run on nearly any computer system.

Is Assembly Language Portable?

- Assembly language is not portable because it is designed for a specific
- processor family.
- There are a number of different assembly languages widely used today, each based on a processor family.

– Some well-known processor families are Motorola 68x00, x86, SUN Sparc, Vax, and IBM-370.

- The instructions in assembly language may directly match the computer's architecture or they may be translated during execution by a program inside the processor known as a microcode interpreter

What you'll learn from Assembly Language Programming

- Some basic principles of computer architecture, as applied to the Intel IA-32 processor family.
- How IA-32 processors manage memory, using real mode, protected mode, and virtual mode.
- How high-level language compilers (such as C++) translate statements from their language into assembly language and native machine code.
- How high-level languages implement arithmetic expressions, loops, and logical structures at the machine level.

What you'll learn from Assembly Language Programming

- You will improve your machine-level debugging skills. – Even in C++, when your programs have errors due to pointers or memory allocation, you can dive to the machine level and find out what really went wrong.
- High-level languages purposely hide machine-specific details, but sometimes these details are important when tracking down errors.
- Assembly language will help you understanding the interaction between the computer hardware, operating system, and application programs.

Applications of Assembly Language

- Embedded systems programs are written in C, Java, or assembly language, and downloaded into computer chips and installed in dedicated devices.
  - Some examples are automobile fuel and ignition systems, air-conditioning control systems, security systems, flight control systems, hand-held computers, modems, printers, and other intelligent computer peripherals
- Many dedicated computer game machines have stringent memory restrictions, requiring programs to be highly optimized for both space and 1runtime speed.
  - Game programmers use assembly language to take full advantage of specific hardware features in a target system.

Virtual Machine Concept

- An effective way to explain how a computer's hardware and software are related is called the *virtual machine concept*.
- Specific Machine Levels

Virtual Machines

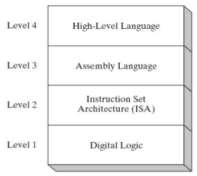
- Virtual machine is a software program that emulates the functions of some other physical or virtual computer.
- Programming Language analogy:
  - Each computer has a native machine language (language L0) that runs directly on its hardware
  - A more human-friendly language is usually constructed above machine language, called Language L1
- The virtual machine **VM1**, can execute commands written in language L1.
- The virtual machine **VM0** can execute commands written in language L0

- Programs written in L1 can run in two different ways:
  - Translation – L1 program is completely translated into an L0 program (which then runs on the computer hardware)
  - Interpretation – L0 program interprets and executes L1 instructions one by one

Translating Languages

- English: Display the sum of A times B plus C.
- C++: cout << (A * B + C);
- Assembly Language:
  mov eax,A
  mul B
  add eax,C
  call WriteInt
- Intel Machine Language:
  A1 00000000
  F7 25 00000004
  03 05 00000008
  E8 00500000

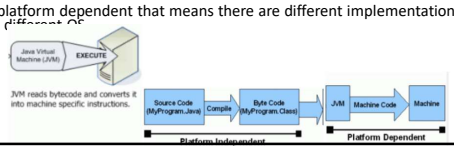
Specific Machine Levels



(descriptions of individual levels follow . . .)

High-Level Language (Level 4)

- Application-oriented languages – C++, Java, Pascal, Visual Basic . . .
- Programs compile into assembly language (Level 3)
- The Java programming language is based on the virtual machine concept.
- A program written in the Java language is translated by a Java compiler into *Java byte code* - a low-level language code.
- Java byte code is executed at runtime by a program known as a *Java virtual machine (JVM)*
- Java code / Bytecode is always the same on different OS.
- That makes java program as platform independent.
- JVM is platform dependent that means there are different implementation of JVM on different OS.



Assembly Language (Level 3)

- Instruction mnemonics that have a one-to-one correspondence to machine language
- Programs are translated into Instruction Set Architecture Level - machine language (Level 2)
- The instructions in assembly language may directly match the computer's architecture or they may be translated during execution by a program inside the processor known as a *microcode interpreter*.

Instruction Set Architecture (ISA) (Level 2)

- Also known as conventional machine language
- Executed by Level 1 (Digital Logic)

Digital Logic (Level 1)

- CPU, constructed from digital logic gates
- System bus
- Memory
- Implemented using bipolar transistors

Self Study

Data Representation

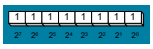
- Binary Numbers
  - Translating between binary and decimal
- Binary Addition
- Integer Storage Sizes
- Hexadecimal Integers
  - Translating between decimal and hexadecimal
  - Hexadecimal subtraction
- Signed Integers
  - Binary subtraction
- Character Storage

Binary Numbers (1 of 2)

- Digits are 1 and 0
  - 1 = true
  - 0 = false
- MSB – most significant bit
- LSB – least significant bit
- Bit numbering:

Binary Numbers (2 of 2)

- Each digit (bit) is either 1 or 0
- Each bit represents a power of 2:



Every binary number is a sum of powers of 2

Table 1-3 Binary Bit Position Values.

2^n	Decimal Value	2^n	Decimal Value
2^0	1	2^8	256
2^1	2	2^9	512
2^2	4	2^{10}	1024
2^3	8	2^{11}	2048
2^4	16	2^{12}	4096
2^5	32	2^{13}	8192
2^6	64	2^{14}	16384
2^7	128	2^{15}	32768

Translating Binary to Decimal

Weighted positional notation shows how to calculate the decimal value of each binary bit:

$dec = (D_{n-1} * 2^{n-1}) + (D_{n-2} * 2^{n-2}) + \ldots + (D_1 * 2^1) + (D_0 * 2^0)$

D = binary digit

binary 00001001 = decimal 9:

$(1 * 2^3) + (1 * 2^0) = 9$

Translating Unsigned Decimal to Binary

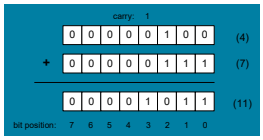
- Repeatedly divide the decimal integer by 2. Each remainder is a binary digit in the translated value:

Division	Quotient	Remainder
37 / 2	18	1
18 / 2	9	0
9 / 2	4	1
4 / 2	2	0
2 / 2	1	0
1 / 2	0	1

37 = 100101

Binary Addition

- Starting with the LSB, add each pair of digits, include the carry if present.



Integer Storage Sizes

Standard sizes:

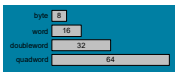


Table 1-4 Ranges of Unsigned Integers.

Storage Type	Range (low–high)	Powers of 2
Unsigned byte	0 to 255	0 to $(2^8 - 1)$
Unsigned word	0 to 65,535	0 to $(2^{16} - 1)$
Unsigned doubleword	0 to 4,294,967,295	0 to $(2^{32} - 1)$
Unsigned quadword	0 to 18,446,744,073,709,551,615	0 to $(2^{64} - 1)$

What is the largest unsigned integer that may be stored in 20 bits?

Hexadecimal Integers

Binary values are represented in hexadecimal.

Table 1-5 Binary, Decimal, and Hexadecimal Equivalents.

Binary	Decimal	Hexadecimal	Binary	Decimal	Hexadecimal
0000	0	0	1000	8	8
0001	1	1	1001	9	9
0010	2	2	1010	10	A
0011	3	3	1011	11	B
0100	4	4	1100	12	C
0101	5	5	1101	13	D
0110	6	6	1110	14	E
0111	7	7	1111	15	F

Translating Binary to Hexadecimal

- Each hexadecimal digit corresponds to 4 binary bits.
- Example: Translate the binary integer 000101101010011110010100 to hexadecimal:

1	6	A	7	9	4
0001	0110	1010	0111	1001	0100

Converting Hexadecimal to Decimal

- Multiply each digit by its corresponding power of 16:
  dec = $(D_3 * 16^3) + (D_2 * 16^2) + (D_1 * 16^1) + (D_0 * 16^0)$
- Hex 1234 equals $(1*16^3) + (2*16^2) + (3*16^1) + (4*16^0)$, or decimal 4,660.
- Hex 3BA4 equals $(3*16^3) + (11 * 16^2) + (10*16^1) + (4*16^0)$, or decimal 15,268.

Powers of 16

Used when calculating hexadecimal values up to 8 digits long:

16^n	Decimal Value	16^n	Decimal Value
16^0	1	16^4	65,536
16^1	16	16^5	1,048,576
16^2	256	16^6	16,777,216
16^3	4096	16^7	268,435,456

Converting Decimal to Hexadecimal

Division	Quotient	Remainder
422 / 16	26	6
26 / 16	1	A
1 / 16	0	1

decimal 422 = 1A6 hexadecimal

Hexadecimal Addition

- Divide the sum of two digits by the number base (16). The quotient becomes the carry value, and the remainder is the sum digit.

```
 36   28   1 28   1 6A
 42   45   58   4B
 78   6D   80   B5
```

21 / 16 = 1, rem 5

Important skill: Programmers frequently add and subtract the addresses of variables and instructions.

Hexadecimal Subtraction

- When a borrow is required from the digit to the left, add 16 (decimal) to the current digit's value:

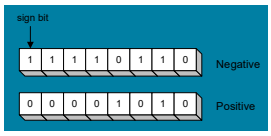
16 + 5 = 21

```
    –1
C6  75
A2  47
24  2E
```

Practice: The address of **var1** is 00400020. The address of the next variable after var1 is 0040006A. How many bytes are used by var1?

Signed Integers

The highest bit indicates the sign.1 = negative, 0 = positive



If the highest digit of a hexadecimal integer is > 7, the value is negative. Examples: 8A, C5, A2, 9D

Forming the Two's Complement

- Negative numbers are stored in two's complement notation
- Represents the additive Inverse

Starting value	00000001
Step 1: reverse the bits	11111110
Step 2: add 1 to the value from Step 1	11111110 +00000001
Sum: two's complement representation	11111111

Note that 00000001 + 11111111 = 00000000

Binary Subtraction

- When subtracting A – B, convert B to its two's complement
- Add A to (–B)

```
00001100          00001100
–00000011   ⟶     11111101
            00001001
```

Practice: Subtract 0101 from 1001.

Learn How To Do the Following:

- Form the two's complement of a hexadecimal integer
- Convert signed binary to decimal
- Convert signed decimal to binary
- Convert signed decimal to hexadecimal
- Convert signed hexadecimal to decimal

Ranges of Signed Integers

The highest bit is reserved for the sign. This limits the range:

Storage Type	Range (low–high)	Powers of 2
Signed byte	−128 to +127	-2^7 to $(2^7 - 1)$
Signed word	−32,768 to +32,767	-2^{15} to $(2^{15} - 1)$
Signed doubleword	−2,147,483,648 to 2,147,483,647	-2^{31} to $(2^{31} - 1)$
Signed quadword	−9,223,372,036,854,775,808 to +9,223,372,036,854,775,807	-2^{63} to $(2^{63} - 1)$

Practice: What is the largest positive value that may be stored in 20 bits?

Character Storage

- Character sets
  - Standard ASCII (0 – 127)
  - Extended ASCII (0 – 255)
  - ANSI (0 – 255)
  - Unicode(0 – 65,535)
- Null-terminated String
  - Array of characters followed by a *null byte*
- Using the ASCII table
  - back inside cover of book

Numeric Data Representation

- pure binary
  - can be calculated directly
- ASCII binary
  - string of digits: "01010101"
- ASCII decimal
  - string of digits: "65"
- ASCII hexadecimal
  - string of digits: "9C"

next: Boolean Operations

Boolean Operations

- NOT
- AND
- OR
- Operator Precedence
- Truth Tables

Boolean Algebra

- Based on symbolic logic, designed by George Boole
- Boolean expressions created from:
  - NOT, AND, OR

Expression	Description
$\neg X$	NOT X
$X \wedge Y$	X AND Y
$X \vee Y$	X OR Y
$\neg X \vee Y$	(NOT X) OR Y
$\neg(X \wedge Y)$	NOT (X AND Y)
$X \wedge \neg Y$	X AND (NOT Y)

NOT

- Inverts (reverses) a boolean value
- Truth table for Boolean NOT operator:

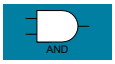
X	¬X
F	T
T	F

AND

- Truth table for Boolean AND operator:

X	Y	X ∧ Y
F	F	F
F	T	F
T	F	F
T	T	T

Digital gate diagram for AND:

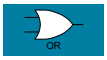


OR

- Truth table for Boolean OR operator:

X	Y	X ∨ Y
F	F	F
F	T	T
T	F	T
T	T	T

Digital gate diagram for OR:



Operator Precedence

- Examples showing the order of operations:

Expression	Order of Operations
¬X ∨ Y	NOT, then OR
¬(X ∨ Y)	OR, then NOT
X ∨ (Y ∧ Z)	AND, then OR

Truth Tables (1 of 3)

- A Boolean function has one or more Boolean inputs, and returns a single Boolean output.
- A truth table shows all the inputs and outputs of a Boolean function

Example: ¬X ∨ Y

X	¬X	Y	¬X ∨ Y
F	T	F	T
F	T	T	T
T	F	F	F
T	F	T	T

Truth Tables (2 of 3)

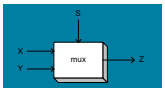
- Example: X ∧ ¬ Y

X	Y	¬Y	X ∧ ¬Y
F	F	T	F
F	T	F	F
T	F	T	T
T	T	F	F

Truth Tables (3 of 3)

- Example: (Y∧S) ∨ (X∧ ¬S)

X	Y	S	Y ∧ S	¬S	X ∧ ¬S	(Y ∧ S) ∨ (X ∧ ¬S)
F	F	F	F	T	F	F
F	T	F	F	T	F	F
T	F	F	F	T	T	T
T	T	F	F	T	T	T
F	F	T	F	F	F	F
F	T	T	T	F	F	T
T	F	T	F	F	F	F
T	T	T	T	F	F	T



Two-input multiplexer

Summary

- Assembly language helps you learn how software is constructed at the lowest levels
- Assembly language has a one-to-one relationship with machine language
- Each layer in a computer's architecture is an abstraction of a machine
  - layers can be hardware or software
- Boolean expressions are essential to the design of computer hardware and software